

MiniPascal Compiler Implementation Report

CSE 303 Compiler Design

Presentation 3 and Final Implementation Report

Najifa Raksanda (202414010)

Swagata Mallick (202414031)

Military Institute of Science and Technology

Department of Computer Science and Engineering

13 September 2026

Abstract

This report examines MiniPascal at main commit 7a57329. Flex and Bison generate the lexer and parser. C code builds an AST, interprets programs, and emits three-address code. Nine TAC tests passed. Main does not generate a separate Pascal executable or perform static type checking. Native generation remains on another branch.

Introduction with background and objectives

A compiler project is easiest to evaluate when its claim is matched to an observable artifact. MiniPascal accepts a deliberately small Pascal-like language and exposes three useful views of an input file: its tokens, its parse tree, and the result of interpreting the tree. The CSE 303 assignment asks for a functional compiler that generates executable code, so the final assessment must distinguish a working front end and interpreter from the missing code-generation stage. We make that distinction throughout this report rather than treating the existence of `minipascal.exe` as proof that a Pascal source file has been compiled into its own executable (Military Institute of Science and Technology [MIST], 2026).

The language includes program headers, variable declarations, scalar and array types, compound statements, assignments, conditional branches, loops, subprogram declarations, calls, arithmetic expressions, Boolean expressions, string literals, and basic input and output. That scope is large enough to demonstrate the main relationships between lexical rules, grammar productions, syntax trees, and runtime behavior, while remaining small enough for a student team to inspect. The source is organized into hand-written Flex, Bison, AST, tree-dump, interpreter, and command-line files. Generated lexer and parser C files are build products rather than source of truth. This separation is consistent with the conventional frontend structure described by Cooper and Torczon (2022).

Presentation 1 established the proposal, and Presentation 2 showed tokens and the parse tree. The pulled main branch adds a textual three-address-code mode. Presentation 3 should therefore distinguish what main implements from what a separate native-backend branch demonstrates. This report describes the main branch at commit 7a57329, the commands used to verify it, and the remaining executable-generation requirement. It also explains why textual TAC is an intermediate representation rather than a runnable program. The Windows x86-64 environment gives a reproducible build context, but it does not decide the storage width or semantics of every MiniPascal variable.

The report uses the assignment brief, repository files from main commit 7a57329, team terminal screenshots, and local execution of the new TAC suite as project evidence. Compiler literature and recent tool documentation support the discussion of parsing, IR, and target representation. The evidence has different strengths: screenshots show particular commands, tests cover selected cases, and code inspection shows how the implementation is organized. The native executable branch is mentioned only as separate development work because it has not been merged into main. This distinction prevents the report from attributing `--compile` behavior to a checkout that currently offers `--tac`, `--run`, `--tokens`, and tree output.

The target machine is a standard Windows desktop or laptop using an x86-64 processor. That describes where the current toolchain runs; it does not determine the size of every MiniPascal variable. The repository's interpreter stores scalar and array numeric values in C double slots. Proposed language-level choices in the earlier presentation included a signed 32-bit integer and a 64-bit real, but that distinction is not enforced by the current runtime. C type names and architecture conventions are separate questions, and explaining them separately avoids the common error of saying that all values are 64-bit simply because the CPU is x64 (Microsoft, 2024, 2025a).

Methodology

The build begins with two specifications that humans edit: `compiler/lexer.l` and `compiler/parser.y`. WinFlexBison converts those specifications into `compiler/lex.yy.c`, `compiler/parser.tab.c`, and a parser header. GCC then compiles the generated files alongside `compiler/ast.c`, `compiler/dump.c`, `compiler/interp.c`, and `compiler/main.c`. The repository provides `build.sh` and `build.ps1` so the same dependency order can be repeated without typing a long command for every demonstration. The program produced by this sequence is `minipascal.exe`. It is the tool that reads Pascal input; it is not a distinct executable generated from `demo.pas` (GNU Project, 2021; GCC, 2025).

The scanner has an explicit rule for each reserved word before its general identifier rule. Its options make keyword recognition case-insensitive and maintain line numbers. The ID expression starts with an ASCII letter or underscore and continues with letters, digits, or underscores. Numeric rules distinguish integer literals from real literals, including a decimal fraction and optional exponent in the real case. The string rule removes the surrounding apostrophes and collapses a doubled apostrophe inside a Pascal string. Operators such as `:=`, `<=`, `>=`, `<>`, and `..` appear before their shorter alternatives, which helps the scanner select the intended token. Whitespace and three comment styles are skipped rather than sent to the parser.

The scanner also maintains a column counter through `YY_USER_ACTION`. For each matched lexeme, the action records the first and last line and column in `yylloc`, then advances the counter by `yyleng`. A newline resets the column. Error branches report an unterminated brace comment, an unterminated parenthesized comment, an unterminated string, or an unexpected character. Location tracking matters because a vague statement that input is invalid is less useful than a message that points to the place where processing failed. GNU Bison's location interface supplies the corresponding positions to grammar actions and diagnostics (GNU Project, 2021).

The grammar declares named tokens for keywords, operators, punctuation, identifiers, and literals. It associates values with identifier strings, integer literals, real literals, and AST nodes through a Bison union. The start symbol is `program`. Representative productions require a `PROGRAM` keyword, a name, a semicolon, optional declarations, optional subprograms, a compound body, and a final dot. Other productions describe type declarations, arrays, parameters, assignment or call statements, `if/else`, `while`, `for`, `return`, and expressions. Precedence declarations put `OR` below `AND`, comparisons below addition, and multiplication above addition. These declarations reduce ambiguity in expressions such as $2 + 3 * 4$ and align the resulting tree with the intended evaluation order (Cooper & Torczon, 2022).

Each successful grammar action creates or connects AST nodes. The `program` action stores the program name, declarations, subprogram list, and compound body in four child pointers. Assignment actions make an `N_ASSIGN` node with a variable target and an expression child. If statements retain their condition and branch nodes. Array types keep lower and upper bounds plus the element type. This is syntax-directed construction: the parser makes a usable structural representation as it recognizes productions. It is different from full semantic validation. The actions do not yet prove that a variable is declared in the right scope or that every operation respects its declared type. Bison actions can compute values or build structures, but the checks chosen for those actions remain the application's responsibility (GNU Project, 2021).

The AST representation uses a `NodeKind` enumeration, line and column fields, a text pointer, a double number field, four child pointers, and a next pointer. The child array holds the fixed relationships inside a construct; next links items in a list. A separate `ast.c` module owns constructors, string copies, list append behavior, and cleanup. `dump.c` traverses the tree and prints labels such as `Program`, `VarDecl`, `Assign`, `Binary`, and `If` with indentation. Keeping the tree printer outside the parser is useful because it can inspect parser output without affecting grammar

recognition. The printed tree is also a teaching artifact: a reader can see that punctuation has served its syntactic purpose and is no longer the main representation (Nystrom, 2021).

The command-line entry point in `main.c` chooses between three modes. `--tokens` calls the scanner directly and prints locations, token names, and lexemes. Default mode calls `yyparse` and prints the tree when an AST exists. `--run` calls the interpreter only when parsing and lexical checks have succeeded. The same `.pas` file can therefore answer different questions without changing the source: what did the scanner see, what structure did the parser build, and what did the runtime do? That division made the demonstrations simpler. It also prevents a syntax-error example from being mistaken for a runtime failure, because the error is produced before interpretation begins.

The runtime uses frames to represent an active call context. A frame points to variable records and, for a function, stores a return value and a link to the previous frame. Variables are found by searching from the current frame outward. Names are compared case-insensitively, matching the language's keyword treatment. A variable record holds a name, an array flag, lower and upper bounds, and a pointer to numeric storage. Arrays allocate one double slot for each index in the declared interval. On access, `lvalue` checks whether the name exists, whether indexing is required or forbidden, and whether an index falls inside the stored range. Those runtime checks prevent some invalid operations even though a separate static type checker has not been written.

Expression evaluation returns a `Val` carrying either a number or a string. Numeric and Boolean literals use the numeric branch; a string literal uses its text branch. Binary operations evaluate their children, then dispatch by operator text. Arithmetic, comparison, and logical operations are supported for numeric values. String equality and inequality compare actual string contents. Other operations with a string operand report that numeric operands are required. The division operators check for zero before dividing. This is a concrete improvement over silently producing an invalid result, and it directly supports the negative test for division by zero (MITRE, 2025).

The statement executor traverses a linked list of AST statements. It handles compound blocks, assignment, calls, `if/else`, `while`, `for`, and `return`. Built-in `read` and `readln` call `scanf` for numeric input. `write` and `writeln` print strings or numbers, adding a newline only for `writeln`. A user-defined function or procedure creates a new frame, maps argument values into parameter variables, declares local variables, executes the body, and restores the caller's frame. The design resembles a tree-walking interpreter: it visits AST nodes at execution time instead of translating the tree to machine instructions ahead of time (Nystrom, 2021).

Error handling is layered. Lexical errors arise in scanner rules, syntax errors arise when the grammar cannot match the token stream, and runtime errors arise after a valid tree is executed. The test cases deliberately exercise more than a happy path. `broken.pas` has a missing semicolon after the program name, so the parser reports line 2, column 1. A separate test places text after the final dot and expects rejection. Another test divides by zero during `--run` and expects both a nonzero process result and an explanatory diagnostic. This layered testing makes it harder for a broad success message to conceal a failure in one stage.

We treated `main` as the authority for currently integrated features and the assignment as the authority for required features. In `main`, `compiler/tac.c` lowers a supported scalar AST to textual three-address code. The `--tac` option prints temporary assignments, labels, conditional jumps, and basic I/O instructions. This is implemented IR, although it is untyped and no static semantic pass precedes it. The command does not invoke GCC or create a separate executable. The native executable generator on `codex/native-executable-generation` is separate branch work and should not be presented as part of `main` until integration and testing succeed. An IR can help separate frontend structure from later backend decisions (Cooper & Torczon, 2022; LLVM Project, 2024).

The platform choice is practical rather than a language guarantee. The team's Windows Git Bash environment uses WinFlexBison binaries and GCC. `x64` defines a hardware and ABI context for the executable running on the machine, while declared MiniPascal types should be specified by the language design and enforced by generated code. At present, `Var.data` is a double pointer and numeric evaluation returns double values. That simplifies a teaching prototype but means an integer declaration does not automatically obtain a distinct signed 32-bit storage slot. A later typed backend would need explicit conversions, range behavior, and layout rules; Microsoft platform documentation is a useful comparison for how C and x64 representation questions differ (Microsoft, 2024, 2025a, 2025b).

The project is stored in Git so source changes, generated outputs, and presentation milestones can be traced. The regression fixes were developed on a dedicated branch and merged into `main` through a pull request. The test suite is available both as a shell script and a PowerShell script, although the PowerShell version needs PowerShell 7 for the expected `stderr` cases in our verification environment. A reproducible claim should include the commit or branch, build command, test command, input file, and observed output. Version control is supporting infrastructure for that evidence, not a replacement for executing the tests (Git Project, 2025).

Implementation details

The first development boundary is between text and tokens. A scanner does not decide whether an entire program is legal; it only labels local pieces of text according to patterns and lexical state. MiniPascal's reserved-word rules return the token identities expected by the grammar, while the identifier rule passes a copied string through `yylval`. That value transfer is important because the parser must keep a program name or variable name after the scanner advances to later input. A literal uses a different member of the Bison union, so the grammar action can distinguish a numeric value from identifier text. In a larger language, this boundary would also handle escape rules and source encodings. The current implementation deliberately narrows identifiers to ASCII letters, digits, and underscore. That is a design choice rather than a property of Pascal in general, and it should appear in language documentation before the group claims broad Pascal compatibility.

Rule order is not merely a formatting matter in `lexer.l`. Flex uses its matching behavior to choose the longest token it can read, and an earlier rule resolves equal-length ties. Placing `program` before the general identifier rule means the text `program` becomes `PROGRAM`, whereas `programCount` can still become an identifier because the longer match wins. The same idea applies to `:=` and `:`, and to the array-range token `..` versus the program terminator `dot`. These examples provide a practical explanation of why a scanner specification is more than a list of token names. If a keyword were placed after the identifier rule, the parser could receive `IDENT` where it requires `PROGRAM` or `BEGIN_KW`, and the error would appear at a later stage. Our arrangement lets the parser operate on the intended vocabulary (GNU Project, 2021).

The comment states deserve separate attention because skipped input still affects locations. The scanner accepts `//` through the end of a line, brace comments, and parenthesized comments beginning with `(*` and ending with `*)`. Two start conditions keep the body of a brace or parenthesized comment from being confused with normal program text. Newlines inside either state reset the column counter, preserving later diagnostics. At end of file within a comment, the scanner increments the error count and reports an unterminated comment. It does not quietly return to normal input. A future test should include comments immediately before a malformed statement so the team can verify that the reported line and column refer to the user's original file, not to a view with comments removed.

Pascal-style quoted text has its own small transformation. In the input language, a doubled apostrophe inside a quoted literal represents one apostrophe in the value. The scanner's

unquote_pascal helper walks the matched text, removes the outer quote characters, and collapses the doubled form. The AST stores the resulting content. That is why a string comparison should operate on the stored string rather than on the source spelling or on a numeric placeholder. The runtime fix uses strcmp for equality and inequality when both operands are strings. An invalid operation such as adding a string to a number is rejected. The current language has no complete declared string type with assignment storage, so this literal handling is useful for output and comparison but should not be presented as a full string-variable implementation.

The program production is the grammar's central contract. It requires the opening PROGRAM token and identifier, a semicolon, optional declarations and subprograms, a compound body, and a final DOT. This gives the top-level syntax a clear completion point. The parser no longer accepts immediately after seeing the dot; it continues far enough to require end of input. That change matters because a source such as program A; begin end. garbage must be rejected, not reported as a successful first program followed by ignored text. The corresponding regression check is small, but it protects an important compiler property: all supplied input should be accounted for. Without it, a user could believe a statement had been compiled when the parser never considered it.

Optional and repeated grammar components are represented as either a null pointer or a linked sequence of nodes. For example, declarations may be absent, while a declaration list connects multiple entries using append. This keeps the grammar readable because productions mirror the source language's sequence structure. It also introduces a memory-management obligation: if parsing fails after a partial list has been built, every allocated name and node should eventually be released. The repository has free_tree for completed ASTs, and main.c calls it after a parse attempt when ast_root exists. Error paths within a parser action merit additional review, especially when a copied identifier has been allocated before a later token fails. A student compiler often works for correct programs first; robust cleanup requires tests with malformed programs that reach deep into nested rules.

Array syntax is a good example of the difference between parsing a declaration and enforcing its meaning. The grammar recognizes ARRAY [integer .. integer] OF a scalar type and makes an N_ARRAY node holding the two bounds and element type. At runtime, make_var computes a storage length from the bounds and lvalue checks each accessed index. The parser does not, however, reject a declaration whose lower bound exceeds its upper bound. The runtime clamps a nonpositive computed length to one slot, which avoids a zero allocation but is not a complete language rule. A semantic pass should reject impossible ranges before allocation. It should also

check that an index expression is integral rather than merely cast a double result to long. Those are concrete next tests, not behavior established by the present demonstration.

Operator precedence in parser.y makes the AST's shape predictable. OR binds less tightly than AND, comparison operators occupy their own nonassociative level, addition follows, and multiplication and division bind more tightly. Unary NOT and signed numeric operators bind at a higher level. A nonassociative comparison rule means chained comparisons need explicit structure rather than accidental evaluation as a sequence of arithmetic results. This is a grammar-level solution to ambiguity. The interpreter then relies on the resulting child arrangement: N_BINOP evaluates the left and right expression nodes and applies the stored operator text. A test of $2 + 3 * 4$ should return 14, while $(2 + 3) * 4$ should return 20. Those examples would specifically connect a parser precedence claim to observed behavior.

The dangling-else form is another grammar issue. The Bison file assigns precedence so an ELSE associates with the nearest unmatched IF in the supported grammar. This choice is common and intuitive, but a presentation should explain it with a nested example if asked rather than simply invoking the word precedence. An AST test could use an outer false condition and an inner condition, then inspect which branch becomes child[2] of which N_IF node. The tree printer would make that association visible. The current automated suite does not contain that case, so the conclusion should be a source-code explanation pending a direct test. This distinction between inspection and demonstration is useful for every nontrivial grammar rule.

The error production under stmt provides a limited recovery path inside statements. When the parser sees an error in a context that can match that production, it creates an empty node and calls yyerror. This can let parsing continue in some situations, but it does not imply that every malformed source produces multiple diagnostics. The broken.pas example fails before the statement region, at the missing semicolon after the program header. The observed output is one syntax error with a location. To make recovery a supported feature, tests would need several independent faults in one program, confirmation that later lines are still analyzed, and a policy for preventing execution when any syntax error occurred. The current main function checks error_count before --run, which helps protect that last boundary.

The AST's fixed child array makes common constructs easy to access, but the meaning of a child index depends on node kind. For N_PROGRAM, child[0] is the name and child[3] is the main body. For N_ASSIGN, child[0] is the target and child[1] is the right-hand expression. For N_IF,

child[0] is the condition and the next positions are branches. That compact representation is appropriate for a course prototype, yet a future maintainer must know the convention for every NodeKind. An alternative would use distinct C structures for each node kind, which improves type clarity at the cost of more code. The report treats the present child layout as a deliberate implementation tradeoff, not a universal compiler architecture (Cooper & Torczon, 2022).

Source positions are copied into AST nodes so diagnostics can refer to the program rather than only to a scanner buffer. Most constructors receive a line from a relevant grammar symbol. The Node structure also has a column field, but the current constructor calls shown in the source are mainly line-oriented. At runtime, messages such as unknown variable and division by zero use the stored line. A further improvement would consistently preserve start and end columns for every AST node, not merely for current scanner tokens. That would let a diagnostic highlight a complete expression or assignment target. The earlier screenshots prove at least one parser location, but they do not prove full-column precision for every runtime error. This is an example of how a modest feature can be evaluated without overstating its reach.

Frame construction and destruction define the lifetime of runtime variables. The interpreter creates a top-level frame, declares globals, executes the main compound body, and frees the frame. A subprogram call adds another frame whose prev link points to its caller. Searching outward allows a nested call to see variables in earlier frames. On return, the old top pointer is restored and the temporary frame is freed. The code also stores the function name and numeric return value in a frame. This mechanism supports the implemented subset, but recursive calls and nested scopes deserve direct stress tests before claiming full Pascal semantics. The fixed argument array of 64 values is another explicit limit; more arguments are not handled through a dynamically sized list.

Input and output are intentionally simple. read and readln use scanf with a double conversion, so the runtime currently expects numeric input for variables. write and writeln evaluate each argument, print string text directly or numeric content in a compact form, and optionally add a newline. Because the renderer prints an integral-valued double with a long format, output can look like an integer even though storage remains double. This detail is relevant to the teacher's question about whether all variables are 64-bit. Printed form, declared Pascal type, C storage type, and CPU word size are four different facts. The runtime screenshot of 5 demonstrates arithmetic and output for one program; it does not settle representation rules for every declared type.

Truth testing is another area where the runtime is broader than a future typed language may allow. The interpreter treats a numeric zero as false and a nonzero number as true; a nonempty string also tests true. That is convenient for a tree walker because a condition can be evaluated with one helper, but a Pascal-like language could require a Boolean expression in if and while. The grammar accepts general expressions in those positions, and the current runtime performs coercion. A static semantic checker should decide whether to preserve that permissive rule or reject non-Boolean conditions. The decision will affect both user-facing examples and backend code generation, so it belongs in a written language specification rather than in an undocumented cast inside eval.

The numeric operators `/`, `div`, and `mod` are handled in one branch, but they do not have identical intended types. The slash operator returns a double quotient. `div` and `mod` cast operands to long and perform integer operations, then convert the result back to double for storage. A zero check occurs on the original double divisor before these conversions. A fractional divisor between negative one and one may cast to long zero even when the original double is nonzero, which deserves a separate guard before integer division. This is a concrete critical-analysis finding from code inspection; the six existing regression checks do not cover it. Before claiming robust arithmetic, the team should add tests for such fractional operands and define whether `div` and `mod` require integer-typed expressions.

Build reproducibility depends on the process environment as well as source files. The team previously installed WinFlexBison outside the repository, so a fresh Git Bash window must have its folder on PATH. GCC was present in the MSYS2 toolchain. `build.sh` checks tool availability before invoking generators and the compiler. If bison or flex is missing, a build failure is a setup error rather than a language error. Presenters should verify `win_bison --version`, `win_flex --version`, and `gcc --version` before the live session, then run the build from the repository root. The report records the tool roles but does not treat a version number shown in a screenshot as a guarantee that every later shell has the same PATH.

The generated C files need an explicit policy in version control. They are reproducible products of `lexer.l` and `parser.y`, but a repository may choose to commit them to simplify builds on machines without generators. This project has generated files in the compiler directory after building. For a clean submission, the team should decide which files are hand-written, which are generated, and whether generated outputs are intentionally tracked. The choice should be documented in README so another student can reproduce `minipascal.exe`. It should also be tested by cloning into a fresh

directory and running only the documented commands. That exercise is more convincing than building repeatedly in a working directory that already contains stale generated files.

Tests are best understood as evidence about a particular revision, not as permanent statements about a project name. The local checks reported here ran in the working clone at its current branch state. The user-facing terminal screenshot came from a separate checkout on main. A merge can make those revisions equivalent, but the report should not infer equivalence from the branch labels alone. Recording `git log -1` and `git status` before the final run would make the relationship inspectable. The Git user manual explains how commits identify repository states, and that principle applies directly to a compiler defense: the examiner needs to know which version produced the displayed output (Git Project, 2025).

Results and outcomes

The supplied terminal evidence shows that `./minipascal.exe demo.pas` prints a parse tree rooted at Program. Below that root, the output identifies the Demo program name, a declaration for `n` as integer, an assignment of 5, and an if whose condition compares `n` with 0. The then branch assigns `n - 1`. This proves that the example was accepted by the lexer and parser and that the AST printer traversed the resulting structure. It does not, by itself, prove that the input was translated into a standalone binary. The distinction matters because a readable AST is evidence for frontend progress, while executable generation requires a different output artifact.

The runtime screenshot supplies a second, separate observation. `show.pas` contains a program header, `begin` and `end`, and `writeln(2 + 3)`. Running `./minipascal.exe --run show.pas` prints 5. The command line and the source listing are visible in the same capture, so the result can be tied to the demonstrated input. The output confirms expression evaluation and the `writeln` path in the current interpreter. It does not show persistent output code. Repeating the program through `minipascal.exe` is still required, which is the practical sign that the program has not been compiled into its own executable.

The negative demonstration uses `./minipascal.exe samples/broken.pas`. The terminal prints parse error at 2:1: syntax error. The sample begins with program Broken without the required semicolon before `var`. The parser therefore reaches the next line while expecting the punctuation prescribed by its program production. The message is evidence of error detection and source location. It is not evidence of full recovery or multiple-error reporting. The grammar contains an error production for

statements, but the observed sample stops at an early program-header error. These distinctions help the group answer a teacher's question about what the diagnostic actually demonstrates.

On 12 September 2026, a local run of `pwsh -NoProfile -File tests/run.ps1` in the working clone reported six successful checks and ended with All tests passed. The checks cover parsing a typed program, rejecting broken.pas, recognizing DOTDOT in an array range, rejecting input after the final dot, reporting division by zero, and evaluating unequal string comparison as false. The shell test script contains the same substantive cases. The terminal screenshot supplied by the team independently shows the parse tree, broken input, and arithmetic output. The reported automated result applies to the checked working clone; the group should run the suite again on the final main checkout immediately before submission.

The token command can show line and column, a token name, and the original lexeme for each instance. The earlier presentation counted 52 named token categories in the lexer and 25 token instances in demo.pas. Those are different quantities. A category such as IDENT can occur many times, so the number of produced tokens depends on the particular input. The distinction is useful when the teacher asks how many tokens the compiler generates: the lexer defines a fixed vocabulary of token types, while any source file produces a variable-length sequence. The syntax tree then groups those token instances into program constructs. The report keeps this earlier count as a repository design fact rather than claiming every input has 25 tokens.

The main build script uses GCC to create minipascal.exe from the compiler's C source, including compiler/tac.c. That binary is the compiler tool. It accepts `--tac demo.pas` and prints textual intermediate code; the displayed instructions are not a new executable. The CSE 303 brief asks for executable generation from a source program, which main still lacks. The separate native-backend branch contains a demonstration of that path, but a branch push alone does not merge it into main. A final integrated defense would need a fresh main build, a new program artifact, direct execution of that artifact, and tests that compare its result with expected MiniPascal behavior.

The numeric storage choice also limits the current result. The interpreter's variable record stores a double pointer, and evaluation expresses most values through double. This allows simple arithmetic and makes the small demonstration work, but it does not implement the proposed type-specific representation of integer, real, Boolean, and char values. An integer may be printed as an integer when its double value has no fractional part, yet that formatting rule does not create a true integer storage type. Boolean truth is tested by numeric zero or nonzero. Declared types are represented in

the AST but are not consistently enforced when values are assigned. This is why the presentation describes static type checking as future work.

Some runtime guards are already present. Unknown variables and procedures generate errors. Array reads and writes check the declared range. A string operand used with an unsupported numeric operator generates a diagnostic. Zero divisors are rejected before `/`, `div`, or `mod`. These checks improve runtime behavior, but they remain dynamic checks encountered only when the relevant path executes. Static analysis would find certain errors before running the program and would provide more precise type rules. The two kinds of checking serve different purposes and should not be collapsed into one claim. The current suite verifies selected cases, not every combination of language features.

The observed successes on main are specific: the tool tokenizes source, parses supported syntax, prints an AST, interprets a small program, and now emits readable scalar TAC. The new Python suite ran nine tests successfully. It checks the slide example, numeric control flow, I/O text, temporary naming, unsupported constructs, syntax rejection, input from the terminal, conflicting modes, and existing token or tree modes. The observed limitations are also specific: TAC is not executable, static type checking is absent, and arrays, subprograms, return statements, and expression calls are unsupported in TAC mode. This account gives the group a defensible feature boundary (MIST, 2026).

Evidence and test interpretation

A useful result table should connect each claim to an input and an expected observation. For `demo.pas`, the command without a mode flag should print a Program-rooted tree. For `show.pas` under `--run`, the output should include 5. For `broken.pas`, the process should exit unsuccessfully and print a parse error at 2:1. For the trailing-input case, success would be a failure of the test because the extra word after the program terminator must be noticed. These cases span different layers, so one cannot replace another. A screenshot of `minipascal.exe` existing on disk proves the C tool was built, but it says nothing about the behavior of a supplied Pascal file. The report uses commands and expected observations together to avoid that category error.

The parse tree is especially valuable because it connects an abstract grammar claim to concrete source. In the demo output, the declaration for `n` is placed beneath the program, the value 5 is beneath an assignment, and the if condition has a binary comparison node. The `n - 1` expression appears as a binary subtraction in the then branch. A teacher can ask why a semicolon or then

keyword is not printed as a separate AST node. The answer is that those tokens helped the parser recognize a production, while the retained tree represents the relationships needed for later interpretation or translation. A parse tree in the formal sense may keep punctuation; this project's displayed structure is more accurately described as an abstract syntax tree.

The arithmetic demonstration deliberately uses a program short enough to inspect in one terminal view. `show.pas` has no declaration, loop, or input dependency; it asks the interpreter to evaluate `2 + 3` and call `writeln`. That simplicity makes the observed 5 easy to explain and hard to confuse with a value carried over from an earlier run. It also means the demonstration covers only one expression and one output call. The group should avoid saying that it proves arrays, functions, all control flow, or generated executables. A strong defense can begin with the small working case, then show separate focused evidence for additional language constructs as those tests become available.

The `broken.pas` output is a useful example of an expected failure. A test suite must treat a rejected invalid program as a pass, and a presenter should say that clearly. The error at 2:1 means the parser detected a mismatch when the next line began, not that the scanner failed to recognize `var` as a token. The original source lacks the semicolon after `program Broken`, so the next keyword arrives in an illegal position. This interpretation can be checked against the program production in `parser.y`. If the group changes the grammar later, the exact location or wording could change; the intended invariant is that malformed input is rejected with a useful source position.

The array-range test looks small but protects an interaction between lexical and grammar rules. In `array [1..5] of integer`, two adjacent dots form `DOTDOT` rather than two `DOT` tokens or part of a real literal. The scanner's real literal pattern requires digits after a decimal point, so `1..5` can be tokenized as `INTEGER_LIT`, `DOTDOT`, `INTEGER_LIT`. The Bison array production expects precisely that shape. A failure at this boundary can appear to be a parser error even though the cause is lexical. The test isolates the tokenization first. It does not test allocation size or runtime indexing; those require separate programs under `--run`.

The trailing-input test reflects the project history. Earlier behavior stopped parsing when the final dot was encountered, which could mask characters that followed a valid program prefix. The current lexer returns `DOT` normally and the top-level Bison parse must reach end of file before reporting success. The regression case puts garbage after a syntactically complete program and expects failure. This matters beyond a course example: a compiler that silently ignores part of its input can produce behavior different from what the programmer believes was submitted. The test

suite now guards that specific boundary, though it should also include comments and whitespace after the dot as legal cases so the rule does not become too strict.

The division-by-zero check exercises an executed path rather than syntax. The input is structurally valid, so scanning and parsing should finish. During expression evaluation, a zero right operand triggers `runtime_error` with the source line and increments the runtime error count. The process then exits unsuccessfully. The test checks both exit status and diagnostic text. MITRE documents divide by zero as a recognized software weakness, but the key project claim is narrower: the implemented interpreter reports this case instead of continuing with a meaningless result (MITRE, 2025).

Boundary cases for `div` and `mod` after numeric conversion remain outside the current test and should be added separately.

The string inequality check protects against a subtler error. If all string values were represented as a numeric zero during comparison, two different strings could appear equal even though their contents differ. The current evaluator has a string branch and uses `strcmp` for equality and inequality. The test evaluates `'a' = 'b'` and expects false, printed as 0. That observation verifies one branch of the comparison logic. A fuller matrix would include equal strings, doubled apostrophes, case differences, an empty string, and mixed string and numeric operands. The intended semantics for mixed operands should be stated explicitly, because the current code treats equality and inequality differently from arithmetic when a string is present.

The earlier PowerShell script checks six lexer, parser, and interpreter behaviors; the newly added Python TAC suite contains nine test methods. In a local copy of main commit 7a57329, the tool built successfully, `--tac demo.pas` printed the expected instructions, and all nine TAC tests passed. Some TAC test methods include multiple input cases, so nine methods is not the total number of programs exercised. Windows PowerShell 5.1 can treat expected standard error as a terminating script error; PowerShell 7 avoids that earlier harness problem. The command `py -3 tests/test_tac.py` is appropriate for the TAC suite when Python 3 is installed.

A check of the current source also reveals cases that can pass parsing but fail only during runtime. An undeclared variable is found by `lvalue` only when an executed expression or assignment tries to access it. An unknown procedure is reported when its call statement executes. An out-of-range array index is caught on access. These are valuable protections, yet a program with an unreachable invalid branch may never trigger them. Static semantic analysis would diagnose such errors before executing any branch. This is the difference between saying the program performs runtime checks

and claiming that it performs comprehensive semantic validation. The latter claim would require a separate AST traversal and tests that show rejection without executing the faulty path.

The executable-generation requirement is distinct from the new TAC result. A .pas input should eventually yield a new program executable that can run without --run. On current main, --tac emits readable instructions such as `n = 5`, `t1 = n > 0`, and `IF_FALSE t1 GOTO L1`. These lines demonstrate lowering from AST to an intermediate form, not machine code or a linked Windows program. The main branch contains no --compile option. A separate branch has a limited native generator, but the team must merge and test it with the TAC changes before claiming both features in the final main submission (MIST, 2026).

The hardware question raised during the previous defense has a similar answer boundary. The program runs on an x64 Windows machine and uses C double storage for numeric MiniPascal variables. The C code's long type is used for some literals, indexing, and integer-like operations, but the Var record stores a double array. Therefore the current implementation does not make every declared variable a 64-bit integer. Nor has it implemented the proposed signed 32-bit integer and 64-bit real representation as two distinct runtime storage classes. Platform ABI documentation can explain the host's calling and data conventions, but language-level sizes must be chosen and enforced by the MiniPascal implementation (Microsoft, 2024, 2025a).

Taken together, the result set supports a measured conclusion. The team has more than a static grammar diagram: the executable tool can analyze a file, report a tree, execute a simple statement, and reject selected invalid cases. It also has less than the final assignment asks for: there is no independent compiled output program, no complete static type pass, and no broad conformance suite. The gap should be shown on a slide and in the report because it changes the meaning of the word compiler in the assessment. The team can still demonstrate solid engineering progress by showing exact inputs, commands, outputs, and code paths, then explaining the next implementation step without inflating the present state.

Reproducibility and remaining verification

A demonstration should start from the repository state that will actually be submitted. The earlier branch used for regression fixes may have the same code as main after merging, but a branch name alone is not proof. Before the defense, the team should switch to main, pull the remote changes, record the current commit, and run the build and test suite again. It should keep one terminal window at the project root so that relative paths in the commands are unambiguous. The program

should receive a named .pas file, since typing into a waiting Git Bash prompt previously caused confusion about end of input. These steps reduce avoidable presentation errors without changing the compiler design (Git Project, 2025).

The current test suite checks representative boundaries rather than the entire language. It does not establish that every nested if, loop, array access, subprogram call, or string literal is correct. An expanded suite should keep each input program in a file, state the expected exit status, and compare both standard output and standard error. For arithmetic, useful cases include precedence, negative operands, real values, and each division operator. For arrays, useful cases include the first legal index, the last legal index, and both out-of-range directions. For control flow, tests should prove that a false branch does not execute and a loop terminates at the intended bound. This is a test plan, not a claim that those cases have already passed.

Type checking needs its own acceptance criteria. A valid assignment should preserve the declared value category, while an incompatible assignment should fail before execution. Function arguments should be checked against parameter count and types. A condition should have a defined Boolean rule. The implementation should also decide whether char is one byte, one Unicode code point, or a restricted ASCII value; the current lexer treats a quoted literal as string text. These decisions should be written down before a backend is added, because generated code must reproduce the same meaning that the interpreter uses. A typed IR can expose that contract explicitly, but only after the language rules are chosen (Cooper & Torczon, 2022; LLVM Project, 2024).

A complete executable-generation test should begin with a valid .pas file and expect a new program artifact distinct from minipascal.exe. The team should then run that artifact directly, without --run and without giving the Pascal source to the interpreter. Current main passes a different test: it emits TAC text for supported scalar input and refuses unsupported input without printing partial TAC. The TAC test harness checks numeric behavior against the interpreter for representative branches and loops. That comparison supports the IR lowering, but it does not establish a working executable backend. The unmerged native branch needs integration before its results can be attributed to main.

The team should preserve terminal commands and inputs beside screenshots so the evidence can be reproduced during defense.

Observed demonstration

The terminal captures below show interpreted output 5 and an intentional syntax error. The TAC listing was reproduced separately from the pulled main branch. A textual instruction listing is evidence of intermediate-code generation; it does not show a separately running Pascal-program executable.

```
$ cat show.pas
program Show;
begin
  writeln(2+3)
end.

USER@DESKTOP-F5KJF91 MINGW64 ~/Documents/Compiler-Theory-Project- (main)
$ ./minipascal.exe --run show.pas
5

USER@DESKTOP-F5KJF91 MINGW64 ~/Documents/Compiler-Theory-Project- (main)
```

Figure 1. show.pas and its observed output 5 under --run.

```
USER@DESKTOP-F5KJF91 MINGW64 ~/Documents/Compiler-Theory-Project- (main)
$ ./minipascal.exe samples/broken.pas
parse error at 2:1: syntax error
```

Figure 2. broken.pas reports a syntax error at 2:1.

Observed TAC from main commit 7a57329

```
.\minipascal.exe --tac demo.pas
n = 5
t1 = n > 0
IF_FALSE t1 GOTO L1
t2 = n - 1
n = t2
L1:
```

Listing 1. Reproduced scalar three-address code for demo.pas.

The branch boundary matters because the reported feature set must match a reproducible checkout. Commit 7a57329 includes tac.c and the --tac option, but its main.c does not recognize --compile. The native branch contains codegen.c and a separate GCC invocation. A student can demonstrate either branch, but cannot claim their combined behavior from the pulled main commit. Before submission, the group should merge deliberately, inspect the resulting command-line modes, rebuild generated lexer and parser files, and run both regression suites. The report should then be revised against the merged commit. Until that happens, the tested main result is textual TAC, not an independent Pascal executable. This distinction also determines which screenshots legitimately support the final submitted report.

Conclusion and critical analysis

MiniPascal on main now includes a lexer, parser, AST, tree printer, interpreter, and scalar TAC emitter. The team can demonstrate the program structure, interpreted output 5, a syntax error with source location, and intermediate instructions for assignments and branches. Nine TAC test methods passed in a fresh local build. This is meaningful progress beyond the earlier frontend, but it does not complete executable generation on main. The separate native-backend branch should be integrated and tested before the report claims that main compiles a .pas program to a standalone .exe. Static type checking and broader feature coverage also remain unresolved.

The strongest part of the current design is the visibility of its stages. --tokens, default tree output, and --run expose different transformations of the same input, which makes failures easier to explain. The source files have fairly clear roles, and recent fixes address trailing text, zero divisors, and string equality. The principal weakness is the mismatch between declared language types and runtime storage. A declaration that says integer is retained in the AST, but the interpreter ultimately stores numeric values as double. The group should not defend proposed 32-bit and 64-bit type widths as a completed property of the implementation. The x64 target only identifies the computer running the tool (Microsoft, 2024, 2025a).

The absence of a static semantic pass has practical consequences. Many errors emerge only if a path executes, and some type mismatches can be accepted by the parser because syntax alone cannot determine whether a particular assignment is meaningful. A symbol table keyed by scope, with declared type and kind, would allow checks for declaration before use, duplicate names, assignment compatibility, function calls, and Boolean conditions. The next version should run that pass over the AST before interpretation or code generation. That sequencing would make generated output more trustworthy and would provide a natural place to attach precise diagnostics (Cooper & Torczon, 2022).

The implemented intermediate representation is a small textual three-address form. compiler/tac.c walks the AST and emits temporary assignments for expressions, labels for control flow, and instructions such as READ, WRITE, and WRITELN. Its output makes the translation of an if statement inspectable: evaluate a condition into t1, jump when false, then execute the assignment before reaching the end label. This representation is useful for explanation and testing, but it is not typed and it is not yet consumed by a backend on main. A future validator and executable generator should agree on the meaning of each instruction (Cooper & Torczon, 2022; LLVM Project, 2024).

For a defense based on the pulled main branch, the team should build the tool, show `demo.pas`, run `-tac demo.pas`, and explain each temporary and branch label. The presenter can also use `--run` to show interpreted output and submit `samples/broken.pas` to demonstrate error reporting. If asked about executable generation, the answer must distinguish current main from the separate branch: main prints IR but does not create a new `.exe`. The native branch has a limited generator, but its behavior is not part of main until a merge and fresh verification. This boundary is more defensible than treating a pushed branch as an integrated release (MIST, 2026).

Priorities for completion

The first priority is an explicit language contract. The parser accepts more constructs than the TAC emitter supports, and the interpreter follows its own runtime rules. A feature matrix should list declared types, legal operations, conversions, scope rules, array bounds, and I/O behavior for each mode. Current TAC handles scalar literals and variables, arithmetic, comparisons, Boolean operations, assignment, if or else, while, for, and basic I/O. It rejects arrays, subprograms, return statements, and calls inside expressions rather than emitting misleading partial output. A static semantic pass should later annotate the AST with types and reject invalid programs before either TAC emission or executable generation.

The second priority is to make the implemented IR a stable contract. Main already emits a simple three-address listing, so the next step is not to invent TAC from scratch. Instead, the team should define typed operations, explicit input and output rules, source locations, and validation for each instruction. A condition such as $n > 0$ becomes a temporary followed by `IF_FALSE` and a label. Tests should check that label placement preserves nested branches and loop behavior. The Python suite already compares representative numeric TAC with interpreter results; stronger cases should cover failure paths and each additional instruction. The IR should remain small enough to inspect during a course defense (Cooper & Torczon, 2022; LLVM Project, 2024).

The third priority is an integrated backend demonstration. The native executable branch provides a separate implementation that translates supported AST nodes directly to C and invokes GCC. Current main, however, emits TAC text and does not consume it to build a program executable. Merging the branches will require resolving their changes to `main.c`, build scripts, and tests. After integration, the team must build from the merged commit, generate a new executable, run it directly, and compare output with the interpreter. They should also explain whether the native backend

consumes TAC or bypasses it; simply having both features in one repository does not make them one pipeline (GCC, 2025).

The fourth priority is differential testing. The nine TAC tests include representative numeric behavior comparisons against the interpreter, including branches, loops, arithmetic, and Boolean expressions. These tests support the textual IR, but the interpreter is not automatically correct. For every case, the expected result should also be reasoned from the source. After native integration, the same input should be run through `--run` and the generated executable, with exact standard output and exit status compared. Invalid programs must be rejected consistently. Useful added cases include array boundaries, strings, nested calls, and type errors when those features are supported in the relevant mode.

The type-storage issue should be resolved before broad code generation. The proposed language has integer, real, Boolean, and char types, but the interpreter's unified numeric slot cannot express each representation faithfully. A signed 32-bit integer has a finite exact range; a 64-bit double can represent many but not all 64-bit integers exactly. A Boolean might occupy a byte in generated C even if its logical domain has only two values, and a char might be an 8-bit code unit or a larger character value depending on the language choice. The x64 machine does not decide these language semantics for the team. A table of chosen widths, conversions, overflow behavior, and I/O formatting should accompany the semantic checker (Microsoft, 2024, 2025b).

The final critical issue is evidence quality. Screenshots are strong when they show both the command and the output, but they are snapshots of one run. A report should record the source input, command line, tool versions, test script, and repository revision so that an examiner can reproduce the claim. Automated tests are stronger for regressions because they compare actual output and status against expected values, yet their scope is only as broad as their cases. The presentation should use a few high-resolution examples and let the report carry detailed procedures. This division respects the audience: a slide explains the result at a glance, while the report lets a technical reader trace how it was obtained and what remains uncertain.

The parser and interpreter also need a clear policy for malformed numeric input. `strtol` and `strtod` convert matched text, but overflow and underflow require explicit checks if the language promises bounded values. An array bound is parsed as an integer literal and later cast into storage arithmetic; a very large range could request an unrealistic allocation. A defensive implementation should validate both bounds, detect multiplication or addition overflow in allocation sizes, and report a

language error rather than relying on malloc failure or host undefined behavior. These cases are outside the visible classroom demo, yet they matter when describing a functional compiler rather than a parser for carefully chosen examples. A small test corpus of boundary literals would make the behavior reviewable without turning the project into a security exercise (MITRE, 2025).

Memory ownership deserves an explicit design note because the scanner, parser, AST, and interpreter pass pointers across module boundaries. The lexer duplicates identifier and string text before returning a token. Grammar actions either transfer those values into a Node or free the temporary copy after creating an owned string. AST cleanup traverses child and next links. The interpreter allocates variable records and frames and releases them when a call or program finishes. That arrangement is workable if each allocation has one owner, but malformed input can interrupt the normal sequence. A leak check on both successful and failing programs would be valuable before the final submission. The code should also avoid using text pointers after their owning AST has been freed. These concerns arise from the present C implementation rather than from the source language's syntax.

Performance is not the main grading target, but it helps explain why this prototype interprets a tree rather than compiling each program immediately. A tree walker repeatedly visits AST nodes whenever a loop executes. The current variable lookup scans linked records from the active frame outward, which is simple and readable for a small class example. A larger program would benefit from symbol resolution before runtime and perhaps indexed storage for local variables. A generated executable could avoid repeated tree traversal and dynamic name searches, but optimizing prematurely would make the existing gap harder to close. The sensible order is to establish correct semantics, generate a basic executable, test agreement with the interpreter, and only then consider performance improvements (Nystrom, 2021; Cooper & Torczon, 2022).

The TAC emitter already reports explicit errors for unsupported AST constructs and avoids printing partial instructions when validation fails. That is a useful diagnostic boundary. The next pass should detect undeclared variables, incompatible assignments, and invalid calls before TAC emission rather than leaving meaning to runtime or a future C compiler. Each diagnostic should refer to the original Pascal source location. If a backend is later connected to TAC, generated-code failures may refer to IR or C lines, so source mapping would make them easier to explain. Good diagnostics should be tested with deliberately malformed and unsupported programs (GNU Project, 2021).

The demonstration environment should be controlled enough that a successful run can be repeated by another student. The repository root contains relative paths such as `demo.pas` and `samples/broken.pas`, so the shell's current directory matters. The WinFlexBison path can change between Git Bash sessions unless added persistently. A fresh clone may not contain generated C files or `minipascal.exe`, depending on version-control choices. These are ordinary setup variables, but they should be written into a short runbook and checked before the presentation starts. The runbook can state the expected prompt directory, generator versions, build command, test command, and the exact example file. If a command fails, the audience can then separate an environment problem from a defect in the language implementation.

Finally, the group should preserve intellectual honesty in its citations. The compiler textbook and tool manuals explain accepted engineering concepts, while the repository and screenshots establish the team's own work. A reference to LLVM does not mean LLVM is part of this implementation; it supports the discussion of intermediate representations as a future direction. A reference to Microsoft x64 conventions does not mean all MiniPascal values use one width; it informs the platform distinction. A reference to a weakness catalog does not prove the project is secure; it explains why a zero-divisor check matters. Keeping each citation close to the claim it supports will help the examiner judge the report's methodological analysis and will avoid making external sources stand in for missing experimental evidence.

This report can guide team handover. Lexer changes need token and parser checks; parser changes need AST, interpreter, and TAC checks. Changes to `tac.c` should run the Python suite because temporary numbering, labels, and I/O instructions are observable outputs. Changes to `interp.c` need successful runtime cases and expected failures. The native backend branch needs its own executable test after merging with main. A pull request should identify the changed behavior and record a fresh build, the relevant test results, and any newly generated artifact. Reviewing the repository branch and commit before a defense prevents screenshots from one branch being attributed to another. That discipline matters more than a feature list that cannot be reproduced from the submitted checkout.

References

Cooper, K. D., & Torczon, L. (2022). *Engineering a compiler* (3rd ed.). Morgan Kaufmann.
<https://www.educate.elsevier.com/book/details/9780128154120>

GCC. (2025). *Using the GNU Compiler Collection (GCC 14.3)*. GNU Project.
<https://gcc.gnu.org/onlinedocs/gcc-14.3.0/gcc/>

Git Project. (2025). Git user manual (version 2.51.0). <https://git-scm.com/docs/user-manual>

GNU Project. (2021). Bison: The Yacc-compatible parser generator (version 3.8.1).
https://www.gnu.org/software/bison/manual/html_node/index.html

LLVM Project. (2024). LLVM language reference manual (LLVM 19.1.0).
<https://releases.llvm.org/19.1.0/docs/LangRef.html>

Microsoft. (2024). C type specifiers. Microsoft Learn. <https://learn.microsoft.com/en-us/cpp/c-language/c-type-specifiers?view=msvc-170>

Microsoft. (2025a). x64 ABI conventions. Microsoft Learn. <https://learn.microsoft.com/en-us/cpp/build/x64-software-conventions?view=msvc-170>

Microsoft. (2025b). Built-in types (C++). Microsoft Learn. <https://learn.microsoft.com/en-us/cpp/cpp/fundamental-types-cpp?view=msvc-170>

MITRE. (2025). CWE-369: Divide by zero. Common Weakness Enumeration.
<https://cwe.mitre.org/data/definitions/369.html>

Military Institute of Science and Technology. (2026). CSE 303 compiler design assignment [Course brief supplied to students].

Nystrom, R. (2021). Crafting interpreters. Genever Benning. <https://craftinginterpreters.com/>

Appendix

The file inventory identifies the hand-written implementation examined for this report. Generated lexer and parser C files are build products. The short excerpts below show representative source and commands; the complete code remains in the project repository.

compiler/lexer.l

Hand-written scanner rules, token values, comments, and source locations.

compiler/parser.y

Grammar productions, precedence, error rule, and AST-building actions.

compiler/ast.h

Node kinds and the public structure used by parser and interpreter.

compiler/ast.c

Node constructors, list links, string copies, and tree cleanup.

compiler/dump.c

Readable tree labels and recursive AST output.

compiler/interp.c

Frames, variables, expression evaluation, statements, and runtime checks.

compiler/main.c

Command-line mode selection and input-file handling.

tests/run.sh

Shell regression checks for parsing, tokenization, and runtime behavior.

tests/run.ps1

PowerShell regression checks with equivalent coverage.

build.sh

Git Bash tool detection and generated-source build sequence.

build.ps1

Windows PowerShell build sequence.

Current main also contains *compiler/tac.h* and *compiler/tac.c* for textual three-address-code generation, plus *tests/test_tac.py* for nine TAC test methods. The verified commands were `.\minipascal.exe --tac demo.pas` and `py -3 tests/test_tac.py`. The TAC output is text; it is not a standalone executable.

Representative MiniPascal input

```
program Show;  
begin  
  writeln(2 + 3)  
end.
```

Representative parser production

```
program : PROGRAM ident SEMI decls subprograms compound DOT  
compound : BEGIN_KW stmts END
```

Reproduction commands

```
sh ./build.sh  
pwsh -NoProfile -File tests/run.ps1  
./minipascal.exe demo.pas  
./minipascal.exe --run show.pas  
./minipascal.exe samples/broken.pas
```